ARTIFICIAL INTELLIGENCE IN GAME

Vuong Ba Thinh
Department of CS –CSE Faculty - HCMUT

Outline

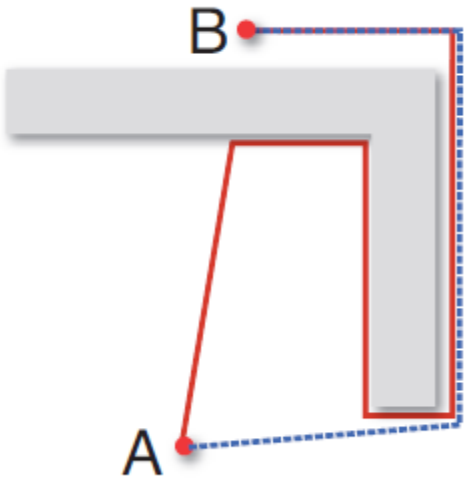
- Real AI versus Game AI
- Path finding
- State-Based Behaviors
- Strategy and Planning

“Real” AI versus Game AI

- Real AI focus on ???
- Game AI focus on ???

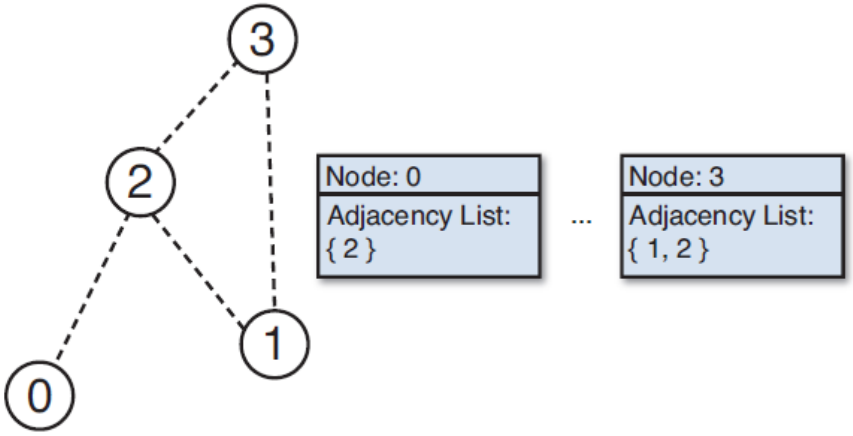
Path finding

- Simple problem: A -> B Intelligently



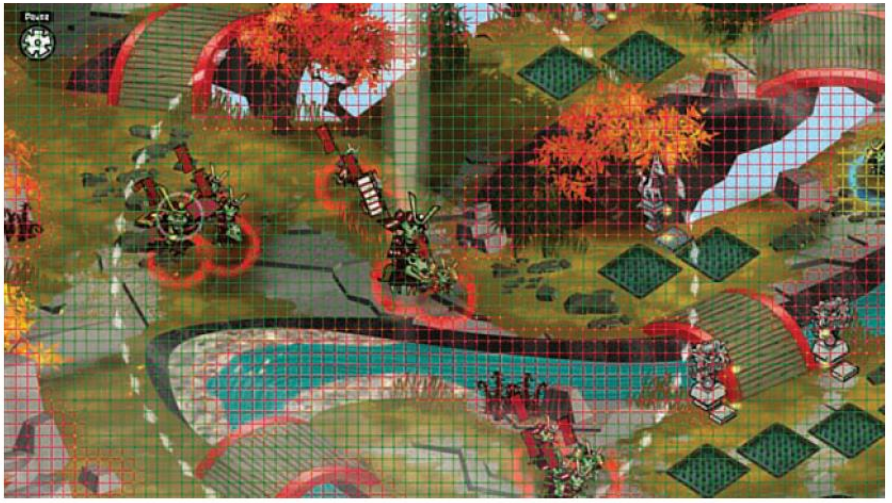
Representing the Search Space

- Turn-based strategy
- Real time action

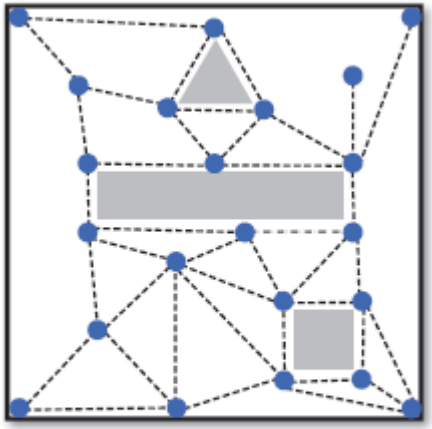


Representing the Search Space

- Grid of squares



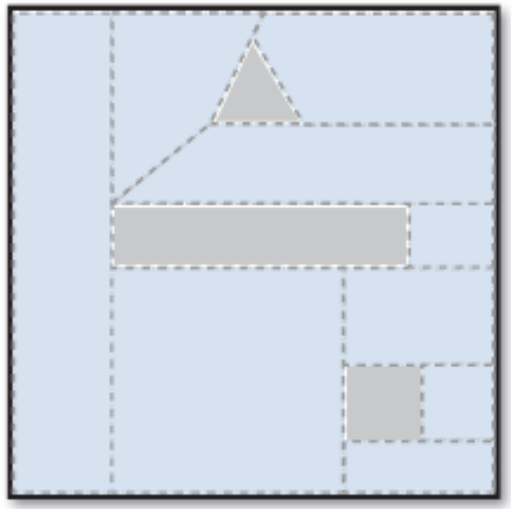
- Path nodes



Path nodes
(22 nodes, 41 edges)

Representing the Search Space

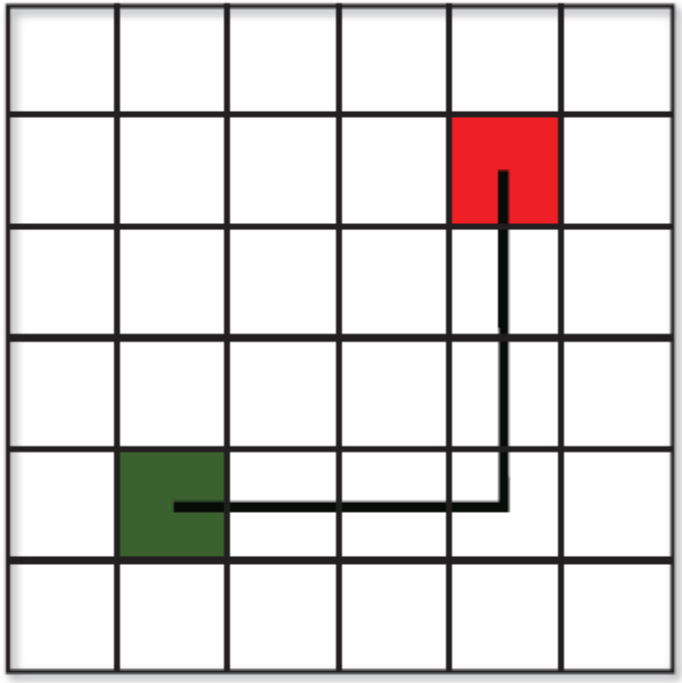
- Navigation Mesh
  - Advantages:
    - ???
    - ???
    - ….



Navigation Mesh
(9 nodes, 12 edges)

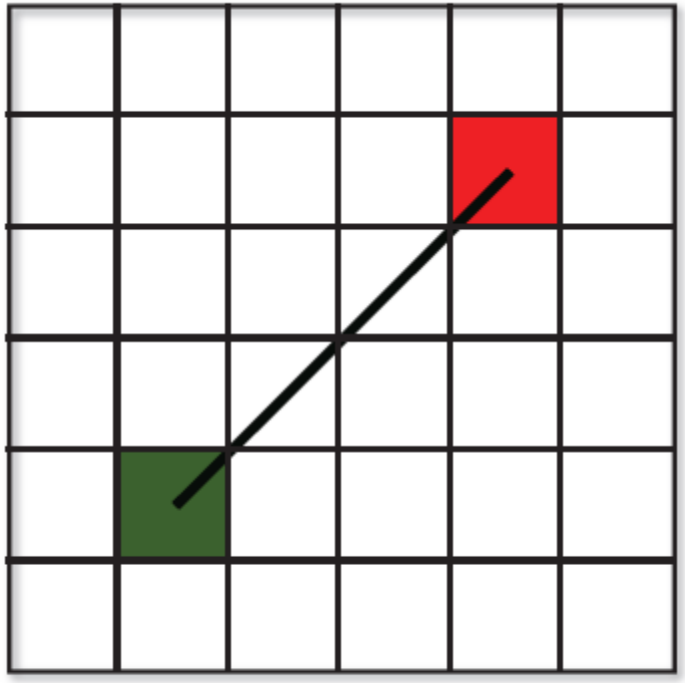
Admissible Heuristics

- h(x): Manhattan distance or Euclidean distance



Manhattan Distance
h(x) = 6

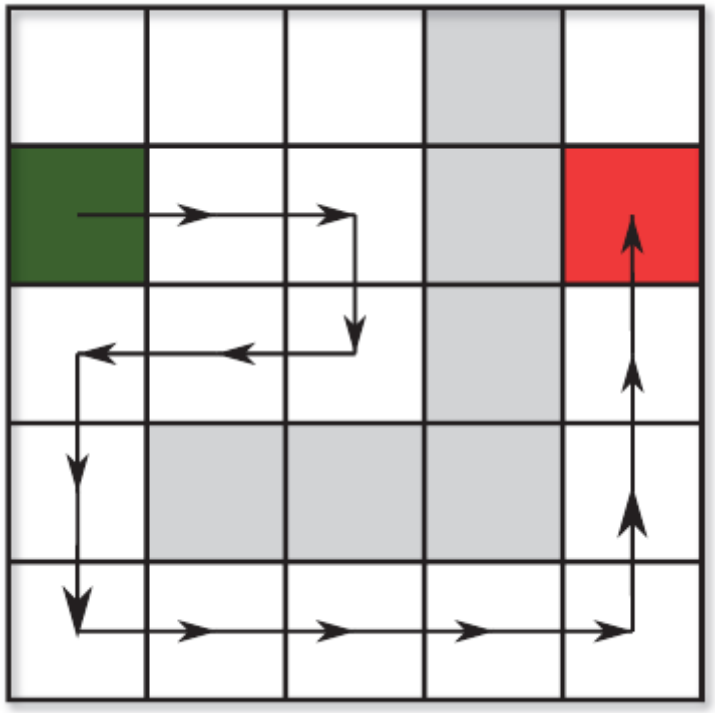
$$h(x) = |start.x - end.x| + |start.y - end.y|$$



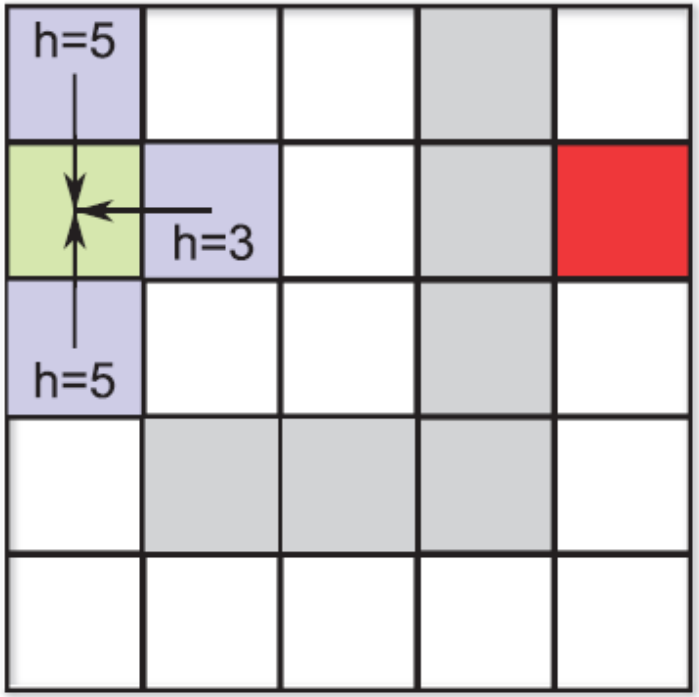
Euclidean Distance
h(x) = 4.24

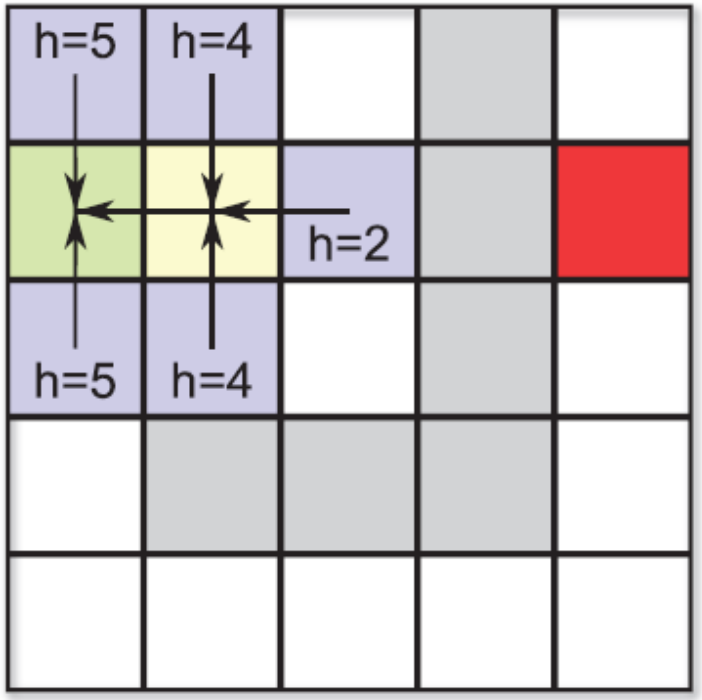
$$h(x) = \sqrt{(start.x - end.x)^2 + (start.y - end.y)^2}$$

Greedy Best First Search



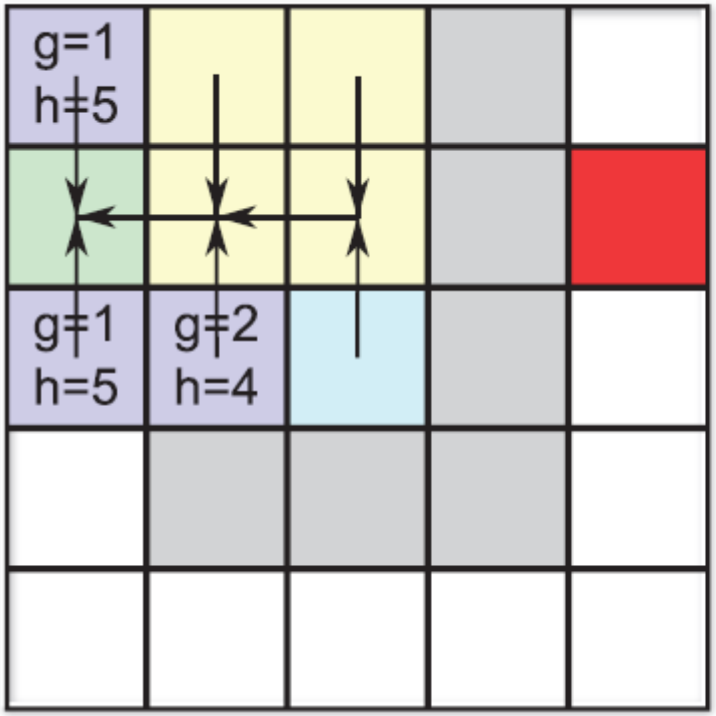
Greedy Best First Search

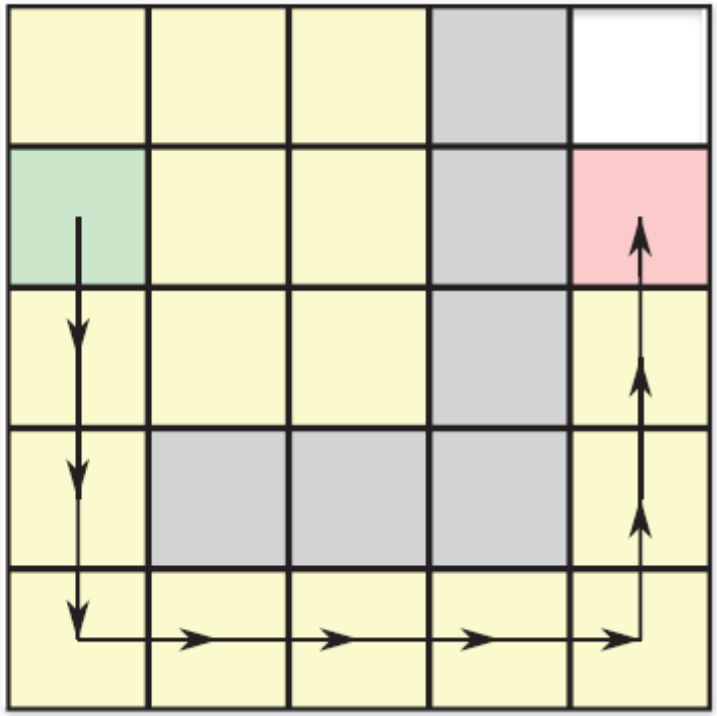




A* Algorithm

- f(x) = h(x) + g(x)





Dijkstra's Algorithm

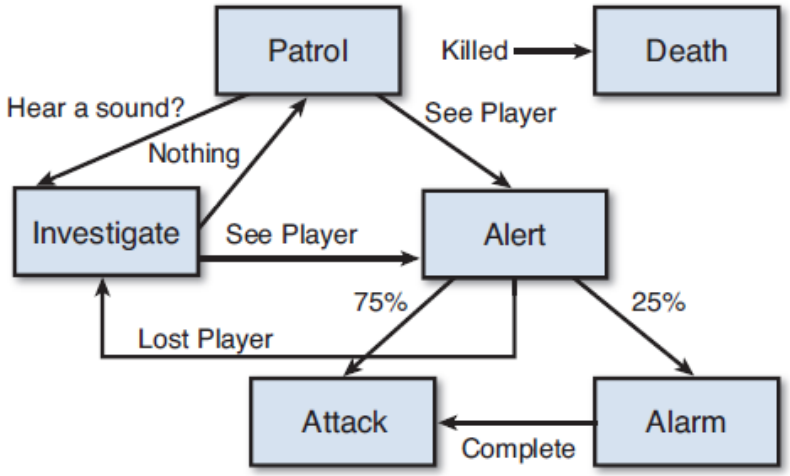
- f(x) = ???

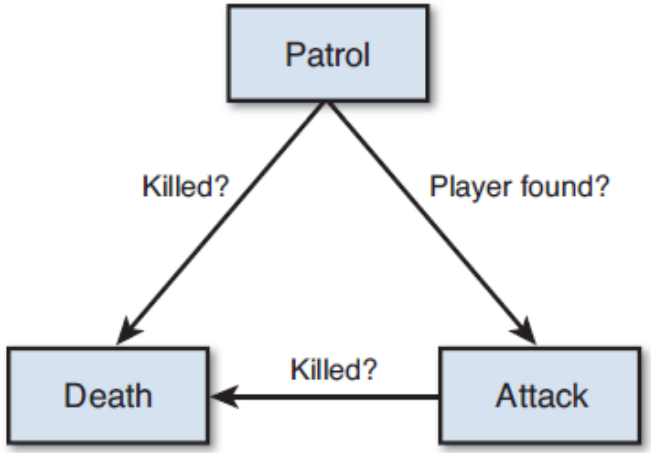
State-Based Behaviors

- State Machines for AI
- Implementation
- State Design Pattern (Reading exercise)

State Machines for AI

- Example: Stealth game





State Machines for AI

- PACMAN: http://gameinternals.com/post/2072558330/understanding-pac-man-ghost-behavior

Implementation

```
enum AIState
    Patrol,
    Death,
    Attack
end
```

```
function AIController.Update(float deltaTime)
    if state == Patrol
        // Perform Patrol actions
    else if state == Death
        // Perform Death actions
    else if state == Attack
        // Perform Attack actions
    end
end
```

Implementation

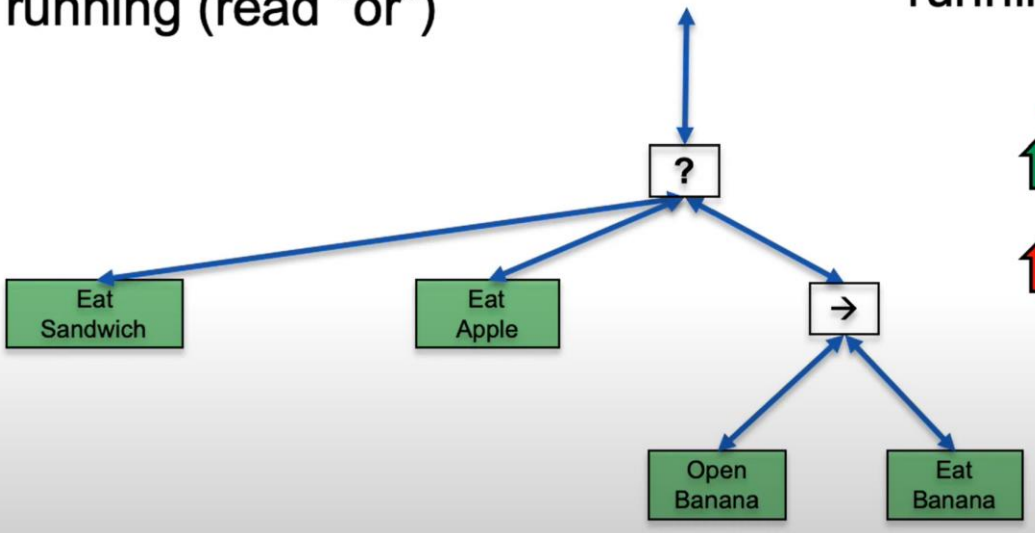
```
function AIController.SetState(AIState newState)
   // Exit actions
   if state == Patrol
      // Exit Patrol state
   else if state == Death
      // Exit Death state
   else if state == Attack
      // Exit Attack state
   end

   state = newState

   // Enter actions
   if state == Patrol
       // Enter Patrol state
    else if state == Death
       // Enter Death state
    else if state == Attack
       // Enter Attack state
    end
end
```

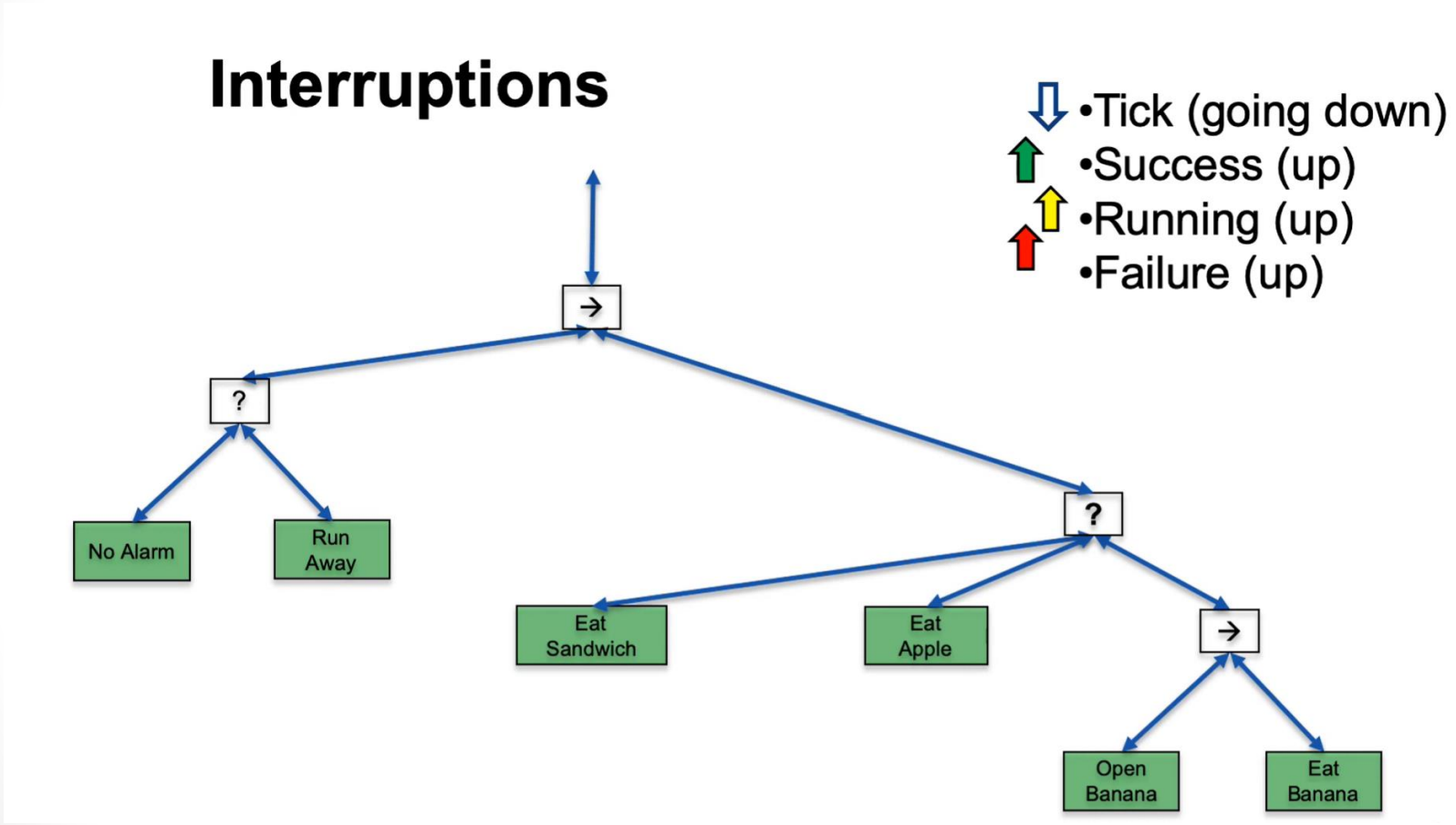
BEHAVIOR TREE

- Fallback
  - Stops at the first child that returns *success* or running (read “or”)
- Sequence
  - Stops at the first child that returns *failure* or running (read “and”)

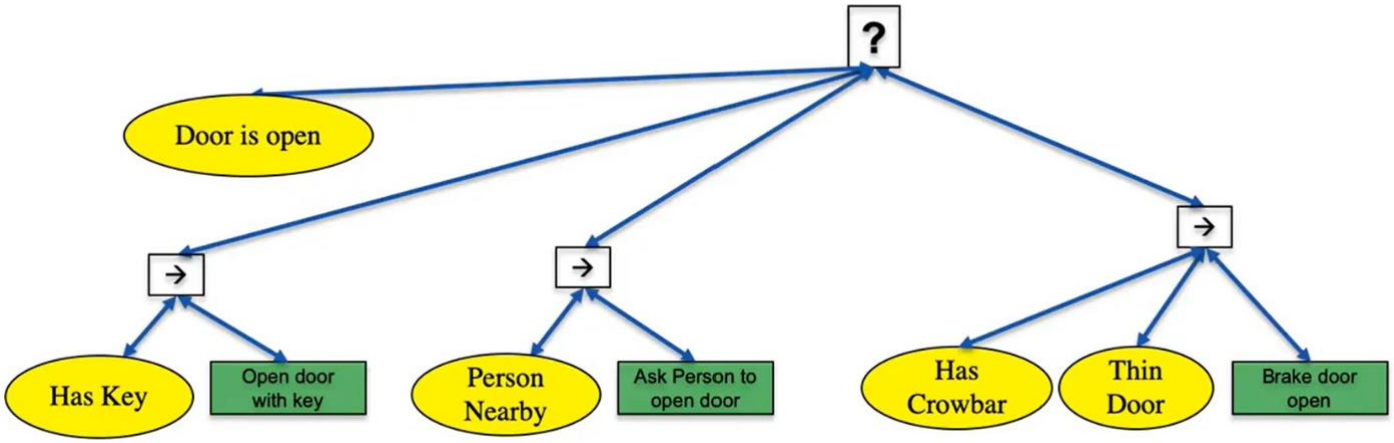


- Tick (going down)
- Success (up)
- Running (up)
- Failure (up)

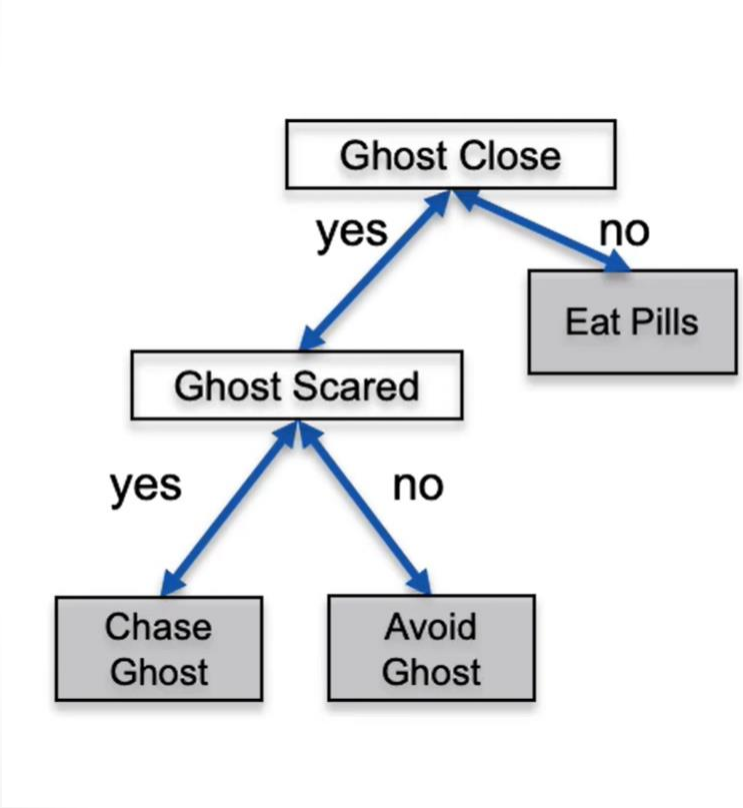
BEHAVIOR TREE

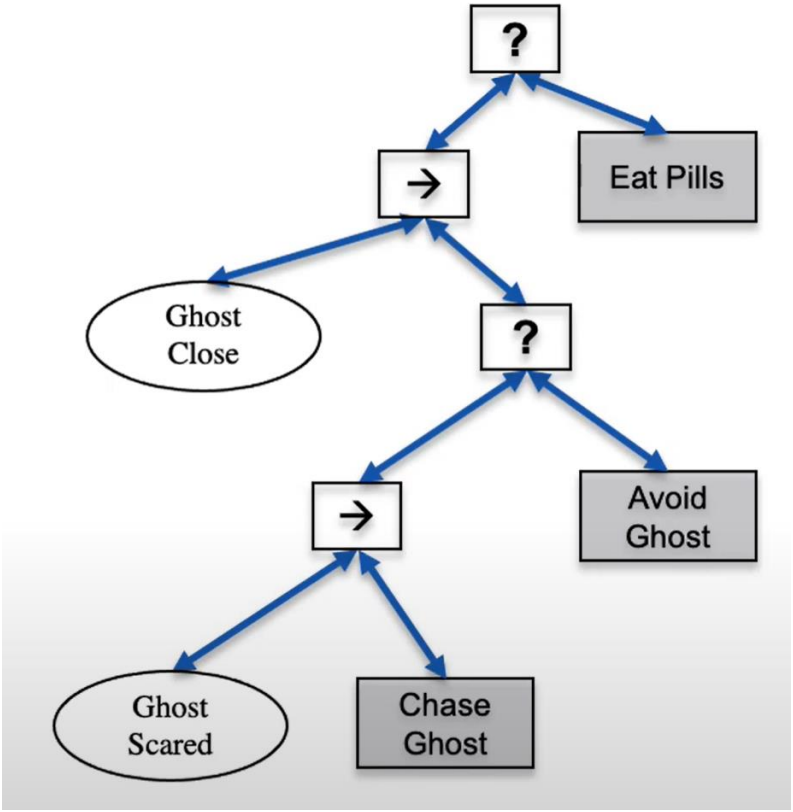


BEHAVIOR TREE



Decision Tree -> BT





Strategy and Planning

- Strategy ???

- Planning ???

More References

- Page 202 in [1]